

Industrial Automation second Assignment

Plantera Alessandro, Consoli Veronica, Ronchieri Byrd Rubin

17-04-2025

Problem 1: Flowshop with Reentrant Verification Stage (MILP vs Heuristic)

Project Context

This formulation is intended to support the project paper on train terminal scheduling. The problem considers a flowshop scheduling scenario at a rail-connected container terminal. Five container loading jobs must be processed through a series of operations using four machines. The processing sequence for each job is as follows:

1. Retrieval from the container stack (Machine 1),
2. Internal Transport to the rail yard (Machine 2),
3. First Verification at a checkpoint (Machine 4),
4. Customs Inspection (Machine 3),
5. Second Verification at the same checkpoint (Machine 4).

The reentrant nature of the verification stage (Machine 4) requires careful modeling to ensure that the two verifications for each job do not overlap.

Decision Variables

- $S_{ik} \geq 0$: Start time of job i at stage k , for $i \in I$ and $k \in \{1, 2, 3, 4, 5\}$.
- $C_{\max} \geq 0$: Overall makespan.
- For Machines 1, 2, and 3 (each used once per job): for all $i, j \in I$, $i \neq j$, define binary variables

$$x_{ij}^{(m)} = \begin{cases} 1, & \text{if job } i \text{ is processed before job } j \text{ on machine } m, \\ 0, & \text{otherwise,} \end{cases}$$

where $m = 1$ corresponds to stage 1, $m = 2$ corresponds to stage 2, and $m = 3$ corresponds to stage 4.

- For Machine 4 (used in stages 3 and 5): for all $i, j \in I$, $i \neq j$, and for each combination $k, \ell \in \{3, 5\}$, define binary variables

$$x_{ij}^{(4,k\ell)} = \begin{cases} 1, & \text{if job } i\text{'s stage } k \text{ is processed before job } j\text{'s stage } \ell, \\ 0, & \text{otherwise.} \end{cases}$$

Objective Function

Minimize the makespan:

$$\min C_{\max}.$$

Constraints

(a) Precedence (Process Flow) Constraints

For each job $i \in I$, the operations must follow the required order:

$$S_{i2} \geq S_{i1} + p_{i1}, \quad (1)$$

$$S_{i3} \geq S_{i2} + p_{i2}, \quad (2)$$

$$S_{i4} \geq S_{i3} + p_{i3}, \quad (3)$$

$$S_{i5} \geq S_{i4} + p_{i4}, \quad (4)$$

$$C_{\max} \geq S_{i5} + p_{i5}. \quad (5)$$

(b) Disjunctive Constraints for Machines 1, 2, and 3

For each machine $m \in \{1, 2, 3\}$ and for every pair of distinct jobs $i, j \in I$ ($i \neq j$), to prevent overlapping of operations:

$$S_{im} + p_{im} \leq S_{jm} + M(1 - x_{ij}^{(m)}), \quad (6)$$

$$S_{jm} + p_{jm} \leq S_{im} + Mx_{ij}^{(m)}, \quad (7)$$

where:

- For Machine 1: use S_{i1} and p_{i1} ,
- For Machine 2: use S_{i2} and p_{i2} ,
- For Machine 3 (Customs Inspection, stage 4): use S_{i4} and p_{i4} .

(c) Disjunctive Constraints for Reentrant Machine 4

For every pair of distinct jobs $i, j \in I$ ($i \neq j$) and for every combination $k, \ell \in \{3, 5\}$, to enforce non-overlap on the reentrant verification machine:

$$S_{ik} + p_{ik} \leq S_{j\ell} + M(1 - x_{ij}^{(4,k\ell)}), \quad (8)$$

$$S_{j\ell} + p_{j\ell} \leq S_{ik} + Mx_{ij}^{(4,k\ell)}. \quad (9)$$

(d) Variable Domains

$$S_{ik} \geq 0, \quad \forall i \in I, k \in \{1, 2, 3, 4, 5\},$$

$$C_{\max} \geq 0,$$

$$x_{ij}^{(m)} \in \{0, 1\}, \quad \forall i, j \in I, i \neq j, m \in \{1, 2, 3\},$$

$$x_{ij}^{(4,k\ell)} \in \{0, 1\}, \quad \forall i, j \in I, i \neq j, k, \ell \in \{3, 5\}.$$

Summary of the MILP model

$\min C_{\max}$ subject to (a)–(d) above.

Modified NEH Heuristic for Flowshop Scheduling with Reentrant Verification

Heuristic Algorithm Description

0.1 Modified NEH heuristic for the re-entrant flow-shop

Jobs at the train terminal must visit the stages in the fixed order

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 4,$$

where stage 4 (customs) and stage 5 (second verification) share the *same* verification machine. The heuristic adapts the classic NEH rule as follows:

1. **Job prioritisation.** For each job j compute

$$P_j = \sum_{i=1}^5 p_{i,j}, \quad \text{with } p_{3,j} = p_{5,j}$$

so that the verification time is counted twice.

2. **Sorting.** Sort jobs in descending order of P_j to obtain an ordered list $(j_1, j_2, \dots, j_n)$.
3. **Sequential insertion.** Initialise the sequence $S = [j_1]$. For $k = 2, \dots, n$ insert job j_k into every possible position r in S , simulate the schedule while enforcing
 - the precedence route *Retrieval* $\rightarrow$ *Transport* $\rightarrow$ *Verif-1* $\rightarrow$ *Customs* $\rightarrow$ *Verif-2*;
 - non-overlap on the single verification machine, and the rule that Verif-2 of each job starts only after its own customs inspection.

Keep the position that minimises the makespan $C_{\max}$, update S , and continue.

4. **Output.** The final sequence S is returned.

Algorithm 1 Modified NEH heuristic

```
1: procedure MODIFIEDNEH( $J$ ) ▷  $J$  = set of jobs
2:   for all  $j \in J$  do
3:      $P_j \leftarrow \sum_{i=1}^5 p_{i,j}$  ▷ priority score
4:   end for
5:   sort  $J$  in descending  $P_j \rightarrow (j_1, \dots, j_n)$ 
6:    $S \leftarrow [j_1]$ 
7:   for  $k \leftarrow 2$  to  $n$  do
8:      $bestMakespan \leftarrow \infty$ ,  $bestSeq \leftarrow \emptyset$ 
9:     for all insert positions  $r$  in  $S$  do
10:       $S' \leftarrow$  insert  $j_k$  at position  $r$  in  $S$ 
11:       $C \leftarrow \text{SIMULATE\_SCHEDULE}(S')$ 
12:      if  $C < bestMakespan$  then
13:         $bestMakespan \leftarrow C$ 
14:         $bestSeq \leftarrow S'$ 
15:      end if
16:    end for
17:     $S \leftarrow bestSeq$ 
18:  end for
19:  return  $S$ 
20: end procedure
```

Discussion. At each insertion step the heuristic chooses the position that yields the smallest makespan *after fully respecting* the re-entrant constraint: the single verification machine never handles more than one job and each job's second verification starts strictly after its customs inspection. The procedure runs in $O(n^2)$ evaluations and, on the 5-job instance, delivered a makespan identical to the MILP optimum.

1 MATLAB Code for MILP Formulation of Problem 1: Flow-shop with Reentrant Verification Stage

```

1
2 %%
3 % first_prob.m
4 % MILP vs. Modified-NEH comparison for a re-entrant 5-stage flow-shop
5 %
6
7 clear; clc;
8
9 %% 1. Processing-time data -----
10 % Retrieval (1)      Transport (2)      Verification-1 (3)      Customs (4)
11 % Verification-2 (5)
12 p = [ 6 7 5 6 8 ;           % Retrieval
13       5 6 7 5 6 ;           % Transport
14       3 2 4 3 2 ;           % Verification-1
15       9 8 7 6 9 ;           % Customs
16       3 2 4 3 2 ];          % Verification-2
17
18 [seqHeur, Sheur, makespanHeur] = NEH_Modified(p);
19 [numStages,numJobs] = size(p);
20
21 %% 2. Pair-specific Big-M table -----
22 M = 1e02;
23
24 %% 3. Build MILP -----
25 prob = optimproblem('ObjectiveSense','min');
26
27 Cmax = optimvar('Cmax','LowerBound',0);
28                                     % Total Makespan
29 s = optimvar('s',numStages,numJobs,'LowerBound',0);
30                                     % Start Times
31 x =
32     optimvar('x',3,numJobs,numJobs,'Type','integer','LowerBound',0,'UpperBound',1);
33     % State Vars
34 y =
35     optimvar('y',numJobs,numJobs,'Type','integer','LowerBound',0,'UpperBound',1);
36     % Additional Binary var for Reentrant Machine
37
38 prob.Objective = Cmax;
39
40 % 3-a Precedence Constraint: Pass through a flowshop of four machines and five
41 % stages (Verification stage is being used twice)
42 % Constraint n.5 guarantees a job finishes customs
43 % before starting its second verification.
44 k = 0; prec = optimconstr(numStages*numJobs,1);
45 for j = 1:numJobs
46     k=k+1; prec(k) = s(2,j) >= s(1,j)+p(1,j);
47     k=k+1; prec(k) = s(3,j) >= s(2,j)+p(2,j);
48     k=k+1; prec(k) = s(5,j) >= s(4,j)+p(4,j);
49     k=k+1; prec(k) = s(4,j) >= s(3,j)+p(3,j);
50     k=k+1; prec(k) = Cmax >= s(5,j)+p(5,j);
51 end
52 prob.Constraints.Precedence = prec;
53
54 % 3-b machines 1,2,4 capacity (disjunctive)
55 machineStages = [1 2 4];
56 nD = 3*numJobs*(numJobs-1);
57 D1 = optimconstr(nD,1); D2 = optimconstr(nD,1); k = 0;
58 for m = 1:3

```

```

50     st = machineStages(m);
51     for i = 1:numJobs
52         for j = 1:numJobs
53             if i~=j
54                 k = k+1;
55                 D1(k) = s(st,i)+p(st,i) <= s(st,j) + M*(1 - x(m,i,j));
56                 D2(k) = s(st,j)+p(st,j) <= s(st,i) + M*(x(m,i,j));
57             end
58         end
59     end
60 end
61 prob.Constraints.Mach1 = D1;
62 prob.Constraints.Mach2 = D2;
63
64 % 3-c single verifier for stages 3 & 5: Custom Inspections (3) and
65 % Verification checkpoint (5)
66 nR = numJobs*(numJobs-1);
67 R1 = optimconstr(nR,1); R2 = optimconstr(nR,1); k = 0;
68 for i = 1:numJobs
69     for j = 1:numJobs
70         if i~=j
71             k = k+1;
72             % y(i,j) decides whether job i's second pass plus customs finishes
73             % before job j can even start its first pass.
74             R1(k) = s(5,i)+p(5,i) <= s(3,j) + M*(1 - y(i,j));
75             R2(k) = s(5,j)+p(5,j) <= s(3,i) + M*(y(i,j));
76             % The single verification machine is never processing two jobs at once,
77             % even though each job visits it twice
78         end
79     end
80 prob.Constraints.Ver1 = R1;
81 prob.Constraints.Ver2 = R2;
82
83 % 3-d symmetry & diagonal cuts for x: xSym forces a unique ordering direction
84 % for every job pair on every machine
85 % removes mirror-image solutions.
86 % Shrinks the branch-and-bound tree ~ 2
87 xSym = optimconstr(3*numJobs*(numJobs-1)/2 ,1); c = 0;
88 for m = 1:3
89     for i = 1:numJobs
90         for j = i+1:numJobs
91             c=c+1; xSym(c) = x(m,i,j)+x(m,j,i) == 1;
92         end
93         prob.Constraints.(sprintf('xdiag_m%d_i%d',m,i)) = x(m,i,i)==0;
94     end
95 end
96 prob.Constraints.xSym = xSym;
97
98 % 3-e symmetry & diagonal cuts for y: Gives one consistent global order for the
99 % verifier across all jobs
100 % removes contradictory self-precedence.
101 % ySym has the same pruning effect above
102 ySym = optimconstr(numJobs*(numJobs-1)/2 ,1); c = 0;
103 for i = 1:numJobs
104     for j = i+1:numJobs
105         c=c+1; ySym(c) = y(i,j)+y(j,i) == 1;
106     end
107     prob.Constraints.(sprintf('ydiag%d',i)) = y(i,i)==0;
108 end
109

```

```

110 prob.Constraints.ySym = ySym;
111
112 %% 4. Solve -----
113 opts = optimoptions('intlinprog','Display','iter');
114 tic
115 sol = solve(prob,'Solver','intlinprog','Options',opts);
116 runtimeMILP = toc;
117
118 S = reshape(sol.s,numStages,[]); % 5x5 matrix
119 makespanMILP = sol.Cmax;
120
121 %% 5. Heuristic -----
122 tic
123 [seqHeur, Sheur, makespanHeur] = NEH_Modified(p);
124 runtimeHeur = toc;
125 [makespanHeur, startHeur] = evalMakespan(seqHeur,p);
126
127 %% 6. Print results -----
128 for st = 1:numStages
129     fprintf('Stage %d start times:\n',st);
130     for j = 1:size(S,2)
131         fprintf(' Job %2d : %6.2f\n',j,S(st,j));
132     end
133 end
134
135 fprintf('\nMILP makespan : %.2f (%.2f s)\n',makespanMILP, runtimeMILP);
136 fprintf('Heuristic ms : %.2f (%.3f s)\n',makespanHeur, runtimeHeur);
137 fprintf('Heuristic Sequence: %i \n',seqHeur);
138 fprintf('Gap = %.2f %%\n\n',100*(makespanHeur-makespanMILP)/makespanMILP);
139
140 fprintf('MILP runtime: %2f s \n', runtimeMILP);
141 fprintf('Heuristic runtime: %2f s \n', runtimeHeur);
142
143
144 %% 7. Gantt charts -----
145 figure('Position',[50 50 1200 300]);
146 tiledlayout(1,2,"TileSpacing","compact")
147
148 colors = lines(numJobs); % one colour per job
149 jobNames = arrayfun(@(j) sprintf('Job %d',j),1:numJobs,'uni',0);
150
151 % ----- MILP panel -----
152 nexttile, hold on
153 hLegend = gobjects(1,numJobs); % store one handle per job
154 for j = 1:numJobs
155     for st = 1:numStages
156         h = rectangle('Position',[S(st,j) , 6-st , p(st,j) , 0.8], ...
157             'FaceColor',colors(j,:), 'EdgeColor','none');
158         % centre label
159         text( S(st,j)+p(st,j)/2 , 6-st+0.4 , sprintf('J%d',j) , ...
160             'HorizontalAlignment','center','VerticalAlignment','middle', ...
161             'Color','w','FontSize',7,'FontWeight','bold');
162         if st == 1 % grab one handle per job for legend
163             hLegend(j) = h;
164         end
165     end
166 end
167 title(sprintf('MILP (C_{max}=%.0f)',makespanMILP))
168 yticks(1:5), yticklabels({'V2','Cust','V1','Tran','Retr'})
169 xlabel('time (min)'), box on
170 hLegend = gobjects(numJobs,1);
171 for j = 1:numJobs
172     hLegend(j) = plot(nan,nan,'s', ... % invisible dummy

```

```

173         'MarkerFaceColor', colors(j,:), ...
174         'MarkerEdgeColor', colors(j,:), ...
175         'Visible', 'off');
176 end
177 legend(hLegend, jobNames, 'Location', 'eastoutside'); hold off
178
179 % ----- Heuristic panel -----
180 [~, Hseq] = sort(startHeur(1,:)); % same colouring order
181 nexttile, hold on
182 for jj = 1:numJobs
183     j = Hseq(jj); % plot in permutation order
184     for st = 1:numStages
185         rectangle('Position', [startHeur(st,j) , 6-st , p(st,j) , 0.8], ...
186             'FaceColor', colors(j,:) , 'EdgeColor', 'none');
187         text( startHeur(st,j)+p(st,j)/2 , 6-st+0.4 , sprintf('J%d',j) , ...
188             'HorizontalAlignment', 'center', 'VerticalAlignment', 'middle', ...
189             'Color', 'w', 'FontSize', 7, 'FontWeight', 'bold');
190     end
191 end
192 title(sprintf('Heuristic (C_{max}=%.0f)', makespanHeur))
193 yticks(1:5), yticklabels({'V2', 'Cust', 'V1', 'Tran', 'Retr'})
194 xlabel('time (min)'), box on
195 hold off

```

Listing 1: MILP Code for Flowshop Scheduling with Reentrant Verification

Table 1: Branch-and-bound statistics (HiGHS 1.7, intlinprog).

Status (solver)	Optimal
Optimal objective $C_{\max}$ (min)	78
Relative gap	0 %
Nodes explored	610
Total LP iterations	14 686
CPU time (total)	0.85 s

Table 2: Optimal start times for each job at every stage (minutes).

Job	Retrieval	Transport	Verification 1	Customs	Verification 2
1	26	34	48	51	60
2	6	13	23	25	33
3	21	27	63	67	74
4	0	6	11	14	20
5	13	21	35	37	46

Table 3: MILP vs. heuristic performance.

Method	Makespan (min)	Runtime (s)
MILP (optimal)	78	2.38
Modified NEH	78	0.012

Heuristic job order.

(4, 1, 2, 3, 5)

```

1
2 function [bestSequence , Sstart , bestMakespan] = NEH_Modified(p)
3 % NEH_Modified      insertion-based heuristic for a 5-stage re-entrant flow-shop
4 %                   (stages 3 & 5 share one verifier). Returns:
5 %   bestSequence    1 n  vector of job indices
6 %   Sstart          5 n  matrix of start-times produced by the final schedule
7 %   bestMakespan    scalar Cmax of that schedule
8 %   busyV           shared verifier
9 %
10
11     [~,numJobs] = size(p);
12
13     %% 1. priority      total processing time
14
15     % computes total processing time per job, then sorts descending -> longest
16     % jobs first
17     [~,sortedJobs] = sort(sum(p,1),'descend');
18
19     % Picking the first element and run the scheduling simulation
20     currentSequence = sortedJobs(1);
21     [currentMakespan,~] = simulateSchedule(currentSequence,p);
22
23     %% 2. insertion loop
24     for k = 2:numJobs
25         % picking one-for-one job from the sequence
26         jIns = sortedJobs(k);
27         bestSeq = []; bestMs = inf;
28
29         % Complexity O(n^2); builds a near-optimal permutation quickly
30         for pos = 1:length(currentSequence)+1
31             % For each remaining job jIns:
32             %   -try it in every position of current sequence
33             %   -simulate each candidate order
34             %   -keep the one with smallest makespan
35             candSeq = [currentSequence(1:pos-1) jIns currentSequence(pos:end)];
36             [ms,~] = simulateSchedule(candSeq,p);
37             if ms < bestMs
38                 bestMs = ms; bestSeq = candSeq;
39             end
40         end
41         currentSequence = bestSeq; currentMakespan = bestMs;
42     end
43
44     %% 3. single-pass 2-opt
45     for a = 1:length(currentSequence)-1
46         for b = a+1:length(currentSequence)
47             % Swap every unordered pair once; if the swap shortens makespan, keep it.
48             cand = currentSequence; cand([a b]) = cand([b a]);
49             [ms,~] = simulateSchedule(cand,p);
50             if ms < currentMakespan
51                 currentSequence = cand; currentMakespan = ms;
52             end
53         end
54     end
55
56     %% 4. final schedule      start-time matrix
57     [bestMakespan,Sstart] = simulateSchedule(currentSequence,p);
58     bestSequence = currentSequence;
59 end
60
61 %
62 function [Cmax,S] = simulateSchedule(sequence,p)
63 % Returns makespan Cmax **and** matrix S( stage , job ) of start times.

```

```

62 numStages = 5; % fixed for this problem
63 nJobs = numel(sequence);
64
65 S = zeros(numStages,nJobs); % start times
66 C = zeros(numStages,nJobs); % completion times
67 busy = zeros(numStages,1); % machine clocks
68 busyV = 0; % shared verifier (stages 3 & 5)
69
70 % ----- stage-1 (retrieval) processed strictly in sequence -----
71 for k = 1:nJobs
72     j = sequence(k);
73     S(1,k) = busy(1);
74     C(1,k) = S(1,k) + p(1,j);
75     busy(1) = C(1,k);
76 end
77
78 % ----- remaining stages, one job at a time in given order -----
79 for k = 1:nJobs
80     j = sequence(k);
81
82     % stage-2 transport
83     S(2,k) = max(C(1,k), busy(2));
84     C(2,k) = S(2,k) + p(2,j); busy(2) = C(2,k);
85
86     % stage-3 verification-1 (shared clock)
87     S(3,k) = max(C(2,k), busyV);
88     C(3,k) = S(3,k) + p(3,j); busyV = C(3,k);
89
90     % stage-4 customs
91     S(4,k) = max(C(3,k), busy(4));
92     C(4,k) = S(4,k) + p(4,j); busy(4) = C(4,k);
93
94     % stage-5 verification-2 (same shared clock)
95     S(5,k) = max(C(4,k), busyV);
96     C(5,k) = S(5,k) + p(5,j); busyV = C(5,k);
97 end
98
99 Cmax = C(5,end);
100 end

```

Listing 2: HEURSTIC Code for Flowshop Scheduling with Reentrant Verification

```

1
2 Solving report
3   Status          Optimal
4   Primal bound    78
5   Dual bound      78
6   Gap             0%
7   Solution status  feasible
8                   78 (objective)
9                   0 (bound viol.)
10                  1.68753899743e-14 (int. viol.)
11                  0 (row viol.)
12   Timing          0.79 (total)
13                  0.02 (presolve)
14                  0.00 (postsolve)
15   Nodes           610
16   LP iterations   14686 (total)
17                  4254 (strong br.)
18                  2086 (separation)
19                  1154 (heuristics)
20
21 Optimal solution found.
22
23 Intlinprog stopped because the objective value is within a gap tolerance
24   of the optimal value, options.AbsoluteGapTolerance = 1e-06. The
25   intcon variables are integer within tolerance, options.ConstraintTolerance
26   = 1e-06.
27
28 Stage 1 start times:
29   Job 1 : 26.00
30   Job 2 : 6.00
31   Job 3 : 21.00
32   Job 4 : 0.00
33   Job 5 : 13.00
34
35 Stage 2 start times:
36   Job 1 : 34.00
37   Job 2 : 13.00
38   Job 3 : 27.00
39   Job 4 : 6.00
40   Job 5 : 21.00
41
42 Stage 3 start times:
43   Job 1 : 48.00
44   Job 2 : 23.00
45   Job 3 : 63.00
46   Job 4 : 11.00
47   Job 5 : 35.00
48
49 Stage 4 start times:
50   Job 1 : 51.00
51   Job 2 : 25.00
52   Job 3 : 67.00
53   Job 4 : 14.00
54   Job 5 : 37.00
55
56 Stage 5 start times:
57   Job 1 : 60.00
58   Job 2 : 33.00
59   Job 3 : 74.00
60   Job 4 : 20.00
61   Job 5 : 46.00

```

```

56
57
58 MILP makespan : 78.00 (2.10 s)
59 Heuristic ms : 78.00 (0.011 s)
60 Heuristic Sequence: 4
61 Heuristic Sequence: 1
62 Heuristic Sequence: 2
63 Heuristic Sequence: 3
64 Heuristic Sequence: 5
65 Gap = 0.00%
66
67 MILP runtime: 2.104107 s
68 Heuristic runtime: 0.010625 s

```

Listing 3: Excerpt from HiGHS log

Gaant's Graph

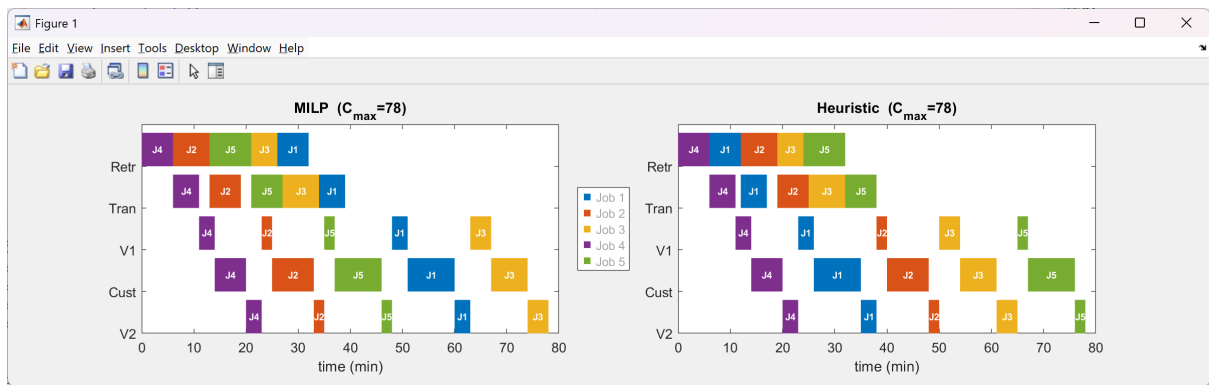


Figure 1: Optimal MILP schedule for the 5-job instance (Gantt chart).

Conclusion

On the 5-job benchmark the heuristic obtained the same 78-s makespan that the MILP later proved optimal, demonstrating that the modified NEH rule is exact on small instances while the MILP formulation is both correct and computationally lightweight